

Sistemas Operacionais

19/02

- Sistema operacional é um programa que aloca e gerencia os recursos de forma eficiente, agindo como intermediário entre o usuário e o hardware:
 - CPU
 - Memória
 - Dispositivos de I/O
- Programa de controle controla a execução dos programas e operações de I/O.
- Kernel - Núcleo de um S.O. que fica entre as aplicações e o hardware (É um programa rodando ininterruptamente)
- Componentes de um sistema:
 - Hardware
 - Sistema Operacional
 - Programas Aplicativos
 - Usuários

26/02

Interrupções:

- Os S.O.s modernos são baseados em interrupções
- Interrupções transferem o controle para a rotina de serviço da interrupção, através do vetor de interrupções
 - Contém os endereços de todas as rotinas de serviço
- Segmentos de código separados determinam qual ação deve ser executada para cada interrupção.

- Exemplo:
 - Tem um processo acontecendo que levará 100 minutos e outro precisa ser feito que levará 5 minutos
 - Uma interrupção pode ocorrer, pausar o primeiro processo, dar a execução para o segundo e esperar ele finalizar
 - Se um temporizador atingir seu tempo limite, algum tratamento acontece

Como o SO, CPU e Outros Processos trabalham juntos?

- Considerando que a CPU consegue rodar um processo por vez
- O SO está no comando de quais processos vão rodar e em qual ordem
- Ele "entrega" para a CPU para um programa rodar seus processos
- Caso outros processos entrem na "fila", um timer (hardware) diz para o SO se o tempo para processar um processo expirou. Quando isso acontece, o SO toma controle e, de acordo com suas políticas, pausa e salva o processo atual e trata o escalonamento das próximas instruções. Posteriormente, ele volta para aquele processo que originalmente estava rodando e esse passo a passo se repete.
 - Mesmo que o primeiro não tenha finalizado, é necessário fazer isso, pois isso gera um paralelismo e permite outro programa também avançar
 - Se não tivesse esse paralelismo, o primeiro programa poderia demorar muito para acabar e causar uma demora excessiva para um outro programa rodar, mesmo que ele seja muito mais rápido
 - Esse paralelismo não é "real". O real acontece quando os dois rodam ao mesmo tempo (Multi-Core), mas o que discutimos acima é uma forma de paralelismo, pois permite mais de um programa avançar

03/03

Proteção de CPU:

- 2 registradores determinam a faixa legal de memória que pode ser acessada pelo programa usuário:
 - Registrador base → Menor endereço físico legal
 - Registrador de limite → Tamanho da faixa legal
 - A memória fora do espaço de endereçamento fica protegida
- Timer (temporizador) → Interrompe o computador após um período de tempo, garantindo o controle do S.O.
 - A cada pulso de clock ele é decrementado
 - Quando o timer atinge o valor 0, uma interrupção ocorre
 - O Timer é utilizado para compartilhar a CPU → Dá tempo para um processo, o timer expira, outro processo roda, o timer expira e retoma o primeiro

Como o usuário executa I/O sendo que ele não tem permissão como o S.O.?

- Permissões de usuário dão a possibilidade do usuário ter uma operação de I/O atendida

Processo:

- Um job e processo são usados como sinônimos e significam que são um programa em execução
- Um processo inclui:
 - Contador de Programa
 - Pilha
 - Seção de Dados

Estados de um processo:

- Novo → O processo está sendo criado:
 - Ainda não está em execução, mas pronto. O processador está ocupado.

- Pronto → O processo está esperando para ser atribuído a um processador
- Em execução → Instruções estão sendo executadas
 - Pode ser finalizado, interrompido ou colocado em espera
- Em espera → O processo espera por um evento
 - O processo estava rodando e passou a esperar algo acontecer, por exemplo uma entrada no teclado
- Encerrado → O processo terminou sua execução

Bloco de Controle de Processos (PCB):

- O S.O. tem uma estrutura encadeada para armazenar uma lista de processos
- Cada processo tem as seguintes informações:
 - Estado do processo
 - Contador de Programa
 - Registradores da CPU
 - Informações de escalonamento de CPU
 - Informações de gerência de memória
 - Informações de contabilidade
 - Informações de status de I/O

Filas:

- Fila de jobs
- Fila de processos prontos
- Filas de dispositivos → Esperando por dispositivos de I/O

17/03

Correção da lista 1

Cheguei atrasado, perdi a 4)

5. As threads ocupam um único espaço de memória, porém cada uma delas tem uma pilha para si mesma. Isso acontece, porque caso fosse uma, não seria possível organizar a ordem de execução de programas. Por exemplo: duas threads rodando programas de forma paralela, cada uma delas tem sua própria pilha de execução, pois se compartilhassem uma, seria difícil demais fazer com que chamadas de função, a fim de exemplo, iniciassem na ordem certa, afinal dois programas estariam um entrando na frente do outro na pilha.

6. $nT - (n-1) \leq P \leq nT$

O tempo de execução de 3 processos de tempo 3, por exemplo, no pior caso seria 9, pois a execução de todos os processos acabariam antes do que estamos analisando. No melhor caso, seria 7, pois ele seria o primeiro a acabar, na última sequência de execuções dos 3.

p1 p2 p3

```
| | |
| | |
| | |
```

Não entendi as próximas questões.

07/04

Correção da lista:

Questão 1)

Passo a passo: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 1, 2, Deadlock

P	C
S	Delay N
1	0 1
0	1 1
1	0 0
0	

Os dois entram na sequência crítica ao mesmo tempo.

Questão 2)

Exclusão mútua;

Posse e espera;

Não preempção;
Espera circular.

Para quebrar o deadlock, podemos matar o círculo.

23/04

Visão lógica de um programa: O endereço pode ser, por exemplo, #0

Visão física de um programa: O endereço #0 é o SO, as demais posições são ocupadas por programas que, por sua vez, possuem suas próprias visões lógicas de endereços.

MMU (Memory Management Unity):

- Unidade que mapeia o endereço virtual em endereço físico.
- Realiza uma soma entre o endereço físico real e o endereço lógico (virtual) da instrução do programa.
 - Com isso, é obtido o endereço real de onde está a instrução.
 - É necessário essa tradução entre lógico e físico.

Alocação de memória:

- Contínua:
 - Partição simples: Dois registradores → Um marca o menor endereço físico. Outro marca o endereço físico limite do processo.
 - Problemas:
 - O S.O. mantém informação de cada bloco de memória sendo utilizado e quais estão livres.
 - Pode ser que a memória fique fragmentada no momento em que um programa acabe.
 - Por exemplo:
P1 (1024 - 2047)
Bloco Vazio (2048 - 3071)
P2 (3072 - 4096)
 - Esse problema acima demanda que o S.O. reorganize a memória em alguns momentos. Compactar a memória gera custo e atrasa as execuções.

30/04

Paginação:

- Quebrar a memória física em blocos de tamanho fixo, chamados quadros (frames).
 - Potência de 2, entre 512 e 8192 bytes.
- Quebrar a memória lógica em blocos de mesmo tamanho, chamados páginas.
- São necessárias tabelas de páginas, que armazenam informações dos endereços virtuais de cada processo.
 - Essas tabelas precisam estar na memória, mas podem estar paginadas também.
- Traz um problema de fragmentação interna.
 - A paginação distribui páginas de forma não necessariamente contígua, eliminando a fragmentação externa.
 - Um processo, normalmente, não ocupa perfeitamente o tamanho de uma página. Por isso, existe um problema de fragmentação interna. Exemplo: Páginas de 4KB e um processo de 10KB. 3 Páginas serão necessárias, onde uma delas tem 2KB sobrando.

21/05

Correção da Lista 6:

- Questão 1

CPU: 50%

I/O: 50%

p1 p2

$$0 \ 0 \rightarrow 0.5 \times 0.5 = 25\%$$

$$0 \ 1 \rightarrow 0.5 \times 0.5 = 25\%$$

$$1 \ 0 \rightarrow 0.5 \times 0.5 = 25\%$$

$$1 \ 1 \rightarrow 0.5 \times 0.5 = 25\%$$

- Questão 2

Tempo efetivo da instrução:

$$(m * k + n)/k$$

Executar a instrução (m) tem um custo de 10. Uma falta de página tem custo de 1000. A cada k instruções, temos uma falta página. Utilizamos a

fórmula acima para encontrar o tempo médio em cada instrução.

- Questão 3
FIFO e LRU padrão.
- Questão 4

09/06

Sistemas de Arquivos:

- **Arquivo:**
 - Espaço de endereçamento contíguo;
 - Tipos:
 - Dados:
 - Numéricos;
 - Caracteres;
 - Binário.
 - Programas.
- **Estrutura de Arquivos:**
 - Nenhuma:
 - Sequência de Bytes.
 - Estrutura de Registro Simples:
 - O arquivo não é apenas uma bagunça de bytes livres, mas sim uma sequência de registros (linhas de uma tabela, por exemplo):
 - Linhas → O S.O. entende que o arquivo é dividido em linhas de texto;
 - Tamanho fixo/variável → O arquivo pode ter registros de tamanho fixo ou variável, de modo que ele saiba onde um começa e outro termina.
 - Estrutura complexa:
 - O S.O. busca compreender a estrutura interna do arquivo por completo:

- Documentos formatados → O S.O. conhece exatamente a divisão interna do arquivo. Por exemplo, alguns sistemas operacionais baseados em registros conhecem exatamente as posições de suas informações, podendo passá-las diretamente pro programa quando necessárias. Nesse modelo, não há necessidade de dar os dados brutos para o programa, e sim o que ele solicitar diretamente pelo S.O.
- Arquivo de carga realocável → É um arquivo executável (ex.: .exe). Para que você clique em um programa e ele rode, o S.O. precisa conhecer uma série de informações, como onde está o código, onde estão os dados e como colocar isso na RAM.
- Quem decide?
 - Se é o S.O. esse controlador, ele conhece as informações dos arquivos, os quais precisam se adequar à estrutura que o sistema suporta;
 - Se é o Programa esse controlador, ele apenas recebe a sequência de bytes recuperada pelo S.O. na memória e, por conta própria, compreende as informações dele.